

Murmur

Swarm-intelligence simulation
for every coding agent

A zero-API-key MCP server that turns the coding subscription you already pay for — Claude, Codex, Gemini, Antigravity, OpenCode, Hermes — into a swarm-intelligence prediction engine. Full five-stage parity with MiroFish, delivered as markdown reports inside your repository. No second bill, no cloud memory lock-in, no web app.

Murmur Project

PRODUCT & ENGINEERING · PLANNING DOCUMENT

SEPTEMBER 2026

Table of Contents

1. Executive Summary 1

2. Market Landscape: MiroFish and the Gap 2

3. Vision, Positioning and Naming 4

4. Target Users and Personas 5

5. Feature Specification: The Five-Stage Pipeline 6

6. Host-Powered Inference: The Zero-Key Architecture 10

7. MCP Tool Catalog v1.0 12

8. System Architecture and Data Model 15

9. Client Support and Distribution 17

10. Technology Stack Decision 18

11. Legal, Licensing and Monetization 20

12. Roadmap, Risks and Success Metrics 21

1. Executive Summary

Murmur is an open-source Model Context Protocol (MCP) server that gives every coding agent the power of swarm-intelligence simulation. It takes the same conceptual engine that made MiroFish a 74,000-star project — ingest seed material, build an ontology and relationship graph, populate a parallel world with persona-driven agents, let them interact on simulated Twitter and Reddit surfaces, then distill the trajectory into a prediction report — and delivers it as a plugin that installs in one line and runs entirely inside the tools developers already live in: Claude Code, Codex CLI, Gemini CLI, Antigravity, OpenCode, Hermes, Cursor, Windsurf, Cline and their peers.

The wedge is economic. MiroFish, in both its open-source deployment and its forthcoming paid online edition, requires users to supply their own **LLM_API_KEY**, a separate **ZEP_API_KEY** for cloud-hosted agent memory, and — for the hosted product — a monthly subscription on top. A developer who already pays for Claude, ChatGPT or Gemini would be paying for artificial intelligence twice to run predictions. Murmur removes the second bill entirely: its engine never calls a model API. Instead, a pattern this plan calls **Host-Powered Inference** lets the coding agent's own subscription-funded LLM perform every generation step, driven by deterministic tools that the Murmur server exposes. No keys, no signup, no cloud memory, no web application.

\$0 extra inference cost beyond the coding subscription already paid	18 MCP tools covering the full five-stage pipeline in v1.0	10+ coding agents supported on day one through one MCP server
--	--	---

Version 1.0 targets full feature parity with MiroFish's five-stage workflow rather than a reduced subset: seed ingestion, ontology generation, graph construction, dual-platform simulation with god's-eye variable injection, report generation, and deep interaction with individual simulated agents after the run. Reports and world state persist as plain files under a **.murmur/** directory inside the repository, which means every simulation is diffable in git, reviewable in pull requests, and reproducible on a colleague's machine.

This document is the combined product requirements and technical plan for v1.0. It sizes the market gap left by MiroFish's paid web strategy, specifies the feature set and MCP tool surface, details the Host-Powered Inference architecture that makes the zero-key promise technically credible, recommends a technology stack through a scored decision matrix, and lays out a six-month roadmap with a risk register. The recommendation, in one sentence: build a lean, clean-room TypeScript MCP server with a local SQLite-backed deterministic engine, ship it on npm, and let the world's existing coding subscriptions become swarm-intelligence engines.

2. Market Landscape: MiroFish and the Gap

2.1 What MiroFish Is

MiroFish describes itself as a simple and universal swarm-intelligence engine for predicting anything. The project presents two faces to the market. The first, [mirofish.ai](#), is the open-source project's home: it links to the GitHub repository, demo videos, and a waitlist for what the team calls the ultimate online edition. The second, [mirofish.my](#), is the commercial front for that online edition — a sign-in-gated console with pricing pages, scenario templates, and a Simulate The Future pitch. The engine itself is genuinely impressive: as of September 2026 the repository holds roughly 74,000 stars, is licensed AGPL-3.0, and has received strategic incubation from Shanda Group.

The engine's five-step pipeline is the conceptual backbone Murmur adopts. First, graph building: seed material is extracted, individual and collective memories are injected, and a GraphRAG knowledge structure is constructed. Second, environment setup: entity relationships are extracted, personas are generated, and agent simulation parameters are configured. Third, simulation: platform-native agents interact across Twitter-style and Reddit-style channels in parallel over multiple rounds while memories update dynamically. Fourth, report generation: a ReportAgent with a rich toolset interacts deeply with the post-simulation environment to produce a readable prediction. Fifth, deep interaction: the user can chat with any individual in the simulated world. Use cases span public-opinion forecasting, policy impact analysis, crisis PR rehearsal, financial sentiment simulation, marketing testing, and literary continuation.

2.2 The Friction Points

For all its engineering quality, every path a MiroFish user can take today carries structural friction that a plugin-native alternative does not. The self-hosted route requires provisioning an OpenAI-compatible LLM API key, a separate Zep Cloud key for long-term agent memory, and operating a Node.js 18+ frontend with a Python 3.11-3.12 backend — three moving parts before the first simulation runs. The project's own quick-start documentation warns that simulations are high consumption and recommends starting with fewer than 40 rounds, which translates directly into third-party API spend for every experiment. The online edition swaps setup friction for a subscription: a second recurring AI bill stacked on top of the coding subscription the same user almost certainly already pays.

Beyond cost, both routes impose a context switch. Prediction work happens in a web application or a locally hosted web UI, away from the terminal and editor where the target audience — people who live in coding agents — actually works. Agent memory lives in Zep Cloud, so simulated worlds leave the user's machine. And because the engine is a standalone web stack rather than a library or protocol service, it cannot be composed with the user's existing tooling: no git diffs between runs, no pull-request reviews of a prediction report, no CI hook that simulates a launch before it ships.

2.3 The Adjacent Field

MiroFish stands on a research lineage rather than alone. Stanford's Generative Agents work (Park et al., 2023) established that LLM-powered agents with memory, reflection and social interaction produce believably emergent group behavior. Projects in the OASIS family scaled that idea toward millions of agents on social-media-like substrates, and MiroFish's own sibling project BettaFish demonstrates the seed-report pattern that feeds prediction engines. Yet everything in this lineage ships as either research code that assumes a GPU cluster and API budget, or as a hosted web product with a subscription funnel. Nothing in the field is shipped as a coding-agent plugin whose marginal cost is zero because it borrows the inference the user has already paid for. That is the unoccupied position this plan targets.

Dimension	MiroFish Online	MiroFish Self-Hosted	Murmur (this plan)
Inference cost	Subscription (pricing unannounced, waitlisted)	Per-token spend on user's own LLM API	\$0 — rides the existing coding subscription
Extra API keys	None for user, service holds them	LLM_API_KEY + ZEP_API_KEY required	None, ever, by design
Agent memory	Vendor cloud	Zep Cloud (external service)	Local SQLite + files inside the repo
Where work happens	Web app, separate account	Local web UI (Node + Python stack)	Inside the coding agent, in the terminal
Outputs	Reports in the web console	Reports in local web UI	Markdown + Mermaid files, git-diffable
Install	Waitlist / signup	Clone, configure env, run two services	One-line npx install per client
License	Proprietary service	AGPL-3.0	Permissive open source (Chapter 11)

Table 2.1 — Option comparison across the dimensions that matter to a coding-agent user



Figure 2.1 — Separate paid dependencies required per option. Murmur's engine performs no model calls, so it adds nothing to the bill.

The market signal is straightforward: paid coding-agent subscriptions are now the default AI budget line for professional developers, and those subscriptions include agentic usage. Claude Pro and Max plans power Claude Code; ChatGPT plans power Codex CLI; Gemini plans power Gemini CLI and Antigravity; open clients such as OpenCode and Hermes ride whatever subscription the user points them at. An engine that converts quota a developer already owns into a capability that previously demanded its own subscription and API keys is not an incremental improvement — it changes who can afford to run swarm simulations at all, from teams with simulation budgets to any individual with a Claude or ChatGPT plan.

2.4 Sources

- MiroFish open-source repository and README: github.com/666ghj/MiroFish (AGPL-3.0; quick-start requirements and workflow description),
- MiroFish project site: mirofish.ai; online edition front: mirofish.my (accessed September 2026),
- Model Context Protocol specification and SDKs: modelcontextprotocol.io;
- Park, J. S., et al. Generative Agents: Interactive Simulacra of Human Behavior. 2023.

3. Vision, Positioning and Naming

Every coding subscription becomes a swarm-intelligence engine. Simulate the future from inside your agent, without a second bill.

3.1 Vision

The long-term vision is that what-if simulation becomes a native verb in every developer's toolkit, the same way code search or test running is today. When a team plans a pricing change, drafts a launch announcement, or writes a policy that will be discussed in public, rehearsing the crowd's reaction should be as cheap and ordinary as running the test suite — one command in the agent, a report in the repository, zero procurement. Murmur exists to make that rehearsal cost nothing beyond the subscription its users already pay, because the marginal cost of inference is the single factor that decides whether prediction stays a specialist activity or becomes an everyday habit.

3.2 Positioning

Murmur positions itself as the IDE-native, zero-marginal-cost alternative to MiroFish's paid web strategy. Where MiroFish asks the market to add a simulation line item, Murmur asks for nothing but an npm install. Where MiroFish's world lives behind a login, Murmur's world lives in the repository — inspectable, diffable, shareable as plain files. The positioning line for launch reads: **upload any report, simulate the future**

instantly, from inside your coding agent — powered by the subscription you already have. The comparison is respectful but pointed: MiroFish proved the engine; Murmur proves the distribution.

Three anti-goals keep the strategy honest. Murmur is not building a web application in v1 — the terminal and the repository are the interface, and a hosted console would reintroduce exactly the costs and context switches the project exists to remove. It is not becoming a model provider or reselling inference — the moment the engine requires its own key, the core promise is broken. And it is not a fork of MiroFish — Chapter 6 and Chapter 11 cover why a clean-room implementation is both an AGPL necessity and an architectural opportunity.

3.3 Naming

The working codename is **Murmur**, with the npm package **murmur-mcp**. The name comes from murmuration — the phenomenon in which thousands of starlings, each following simple local rules with no leader, produce complex emergent patterns across the whole flock. That is a precise description of the product: many simple agents, local rules, emergent collective behavior worth predicting. The word is short, pronounceable, verb-able (murmur a simulation), and unclaimed in the MCP ecosystem as of this plan. The name remains a placeholder until a trademark screen is completed in Milestone 0; nothing in the architecture depends on it.

4. Target Users and Personas

Murmur serves a horizontal audience — developers, product and growth teams, analysts, and creative writers — through one engine differentiated by scenario templates rather than by separate products. The adoption path is deliberately developer-first, not because other users matter less, but because developers are the population that installs MCP servers, and every installed server is a gateway through which the other personas arrive: the developer installs Murmur, then the product manager asks for the launch template, then the analyst asks whether the same world can be re-run with a policy variable changed. Personas below therefore describe both the user and the template pack that meets them.

Dana — the senior developer living in Claude Code

Dana maintains a popular open-source library and is about to publish a breaking release with a rewritten pricing page for the hosted tier. Her trigger is anxiety about the community reaction: will Hacker News and r/programming read the migration guide kindly or as a betrayal? Today she guesses, or asks one colleague. With Murmur she runs the launch template from her agent: the README, changelog and pricing page are already in the repo, the world seeds itself from those files, and twenty minutes later she is reading a report on the likely top complaints — including one misreading of the license change she had not anticipated, which she fixes before shipping. Her success metric: every major release is preceded by a simulation report committed alongside the release notes.

Priya — the product manager rehearsing a launch

Priya owns pricing and messaging for a SaaS product. Her trigger is a coming change — new pricing tiers, a rebrand, a deprecation — that will be discussed in public within days. She does not live in a terminal, but her team does. She asks the team's agent to run the launch-reaction template with the draft announcement and FAQ as seeds, then iterates: inject a competitor undercutting the price mid-simulation, re-run, compare the two reports. What used to be a gut-feel decision becomes a rehearsed one, with the top misunderstanding risks written down before the post goes live. Her success metric: the top three predicted objections from the report appear in the pre-emptive FAQ she publishes.

Marcus — the analyst who needs reproducibility

Marcus works in policy or financial analysis and needs scenario runs he can defend: same seeds, same parameters, same report. His trigger is a stakeholder asking what happens to public opinion if a draft policy is announced next month versus next quarter. Proprietary tools are expensive; research codebases demand an API budget he does not control. With Murmur, a run is a directory: seeds, world configuration, generated posts, and the final report are all files under `.murmur/`, so he can version them, diff two scenarios, and attach the whole world to a memo. His success metric: a colleague reproduces his headline finding from the committed files alone, on a different machine, without him in the room.

Leo — the creative writer exploring a story world

Leo writes serial fiction and treats Murmur as a sandbox for what-ifs: he seeds a world with two chapters, instantiates his characters as personas with their motives, and asks what happens after the funeral scene if the eldest daughter reveals what she found in the study. The simulated fandom argues about motivations in a Reddit-style thread; the report surfaces which relationship the crowd believes will break first, which may or may not be the one Leo intended — and that gap between author intent and reader reception is exactly what he wants to see. His success metric: Murmur sessions become part of his drafting loop, the way a whiteboard is.

5. Feature Specification: The Five-Stage Pipeline

Version 1.0 commits to full parity with MiroFish's five-stage workflow. The stages map cleanly onto a plugin interaction model: each stage is a small set of MCP tools the host agent orchestrates, and every stage persists its output into the world directory so a session can be paused, resumed, inspected or re-run at any stage boundary. This chapter specifies the seven feature groups F1-F7, each with its user story, behavior, acceptance criteria and a parity note against MiroFish's corresponding step.

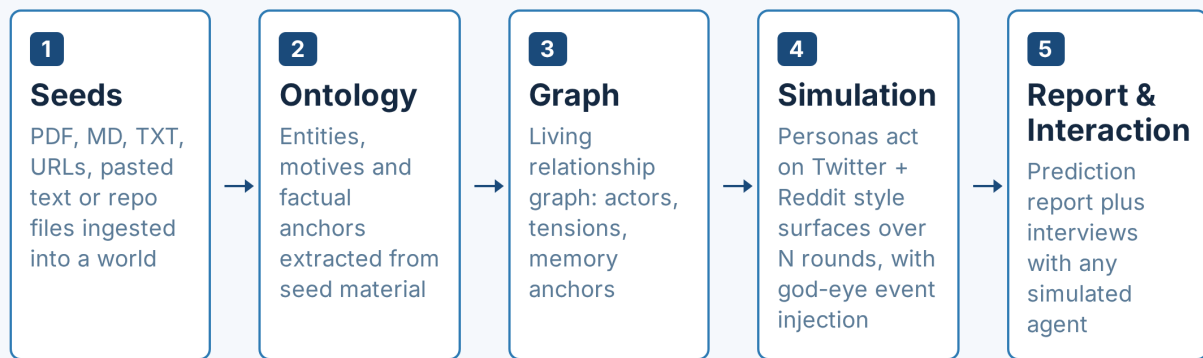


Figure 5.1 — The five-stage pipeline as users experience it. Stages 1-3 build the world; stage 4 evolves it; stage 5 reads it out and opens it for interrogation.

F1 — World and Seed Ingestion

User story. As any persona, I point my agent at material — files in the repo, a URL, pasted text — and a simulated world is created from it without configuration. The server accepts PDF, Markdown and plain-text files up to 50 MB total per world (matching MiroFish's documented upload envelope), raw URLs fetched by the host agent and passed in as text, and pasted snippets. A world is created with `world_init`, seeds are attached with `seed_add_files` / `seed_add_url` / `seed_add_text`, and `seeds_review` returns a digest the host can show the user before proceeding. **Acceptance:** a new user with Claude Code and a 10-page PDF reaches a seeded world in under two minutes; seeds are deduplicated and byte-hashed so identical material is never ingested twice. **Parity note:** matches MiroFish's upload console, minus the web form, plus direct access to repository files as first-class seeds.

F2 — Ontology Generation

User story. As a user, I want the raw seed material turned into structured entities, motives and factual anchors without hand-labeling. `ontology_plan` returns a batched extraction task — a compact prompt bundle containing the seed digests and a strict JSON schema — which the host LLM completes in one call; `ontology_submit` ingests and validates the result against the schema, then writes entities, motives and anchors into the world store. Failures are validated, itemized and retried individually rather than re-running the whole batch. **Acceptance:** on the reference corpus (a public incident brief plus policy memo), extraction yields 10-30 entities with at least 90 percent schema-valid fields on first submit. **Parity note:** corresponds to MiroFish's reality-seed extraction step; the difference is that the extraction intelligence is the host agent's model, not a server-side API call.

F3 — Graph Construction

User story. As a user, I want a living relationship graph that exposes the actors, tensions and memory structure behind the scenario. `graph_build` derives typed relations between ontology entities (alliance, opposition, influence, ownership), scores tensions, and attaches memory anchors — the facts each entity is presumed to know at simulation start. `graph_query` traverses the graph (by entity, relation type, or tension

threshold) and returns compact sub-graphs sized for the host's context window; `graph_export_mermaid` renders the whole graph or a sub-graph as a Mermaid block committed to the world directory, viewable on any git forge. **Acceptance:** a 15-entity world produces a Mermaid diagram that renders cleanly in GitHub markdown; `graph_query` responses stay under 4,000 tokens for default depth. **Parity note:** matches MiroFish's GraphRAG construction step, with SQLite-backed local storage in place of the GraphRAG service.

F4 — Persona and Environment Setup

User story. As a user, I want a believable population — personas with demographics, dispositions, motives and platform habits — generated to match the scenario, in whatever size I choose. `personas_plan` returns a persona-drafting task (count, archetype distribution derived from the ontology, per-platform assignment); `personas_submit` validates and stores the population. Personas carry a compact Big-Five-lite trait vector, a stance toward each salient entity, an activity level and a platform preference (Twitter-style quick reactions, Reddit-style argumentative threads, or both). `persona_inspect` and `persona_edit` let the user audit and fix any individual before the run — the audit step MiroFish's batch pipeline lacks. **Acceptance:** populations from 8 to 128 personas are supported in v1, with default 24; each persona renders in under 120 tokens of context. **Parity note:** matches MiroFish's persona generation and environment config, adding explicit per-persona editability because an agent-mediated interface makes it cheap.

F5 — Dual-Platform Simulation with Variable Injection

User story. As a user, I want to press play and watch the world evolve over multiple rounds — and to reach in mid-run with a god's-eye event. `sim_configure` sets rounds (default 8, max 40 — the same practical ceiling MiroFish's documentation recommends), platforms, posting rates and virality parameters. Each round is a batched exchange described fully in Chapter 6: `sim_next_batch` composes a generation task for the agents activated this round (each persona sees a personalized digest of the current feed, its own memory, and its stance vector); the host LLM produces the posts, replies and votes; `sim_submit_generations` ingests them, updates memories and feeds, and computes engagement. `sim_inject_event` drops an external event into the world between rounds — a competitor price cut, a leaked memo, a celebrity opinion — and the next round's agents react to it. `sim_round_summary` and `sim_timeline` give compact readouts of any round or the whole trajectory. **Acceptance:** an 8-round, 24-persona world completes in one continuous agent session on Claude Code with no context overflow; injecting an event at round 5 measurably shifts the sentiment trajectory in the round-6 summary. **Parity note:** matches MiroFish's dual-platform parallel simulation and dynamic temporal memory updates; the platform mechanics (feed ranking, threading, virality) are reimplemented deterministically in the engine rather than simulated by prompt.

F6 — Report Generation

User story. As a user, I want the trajectory condensed into a decision-ready prediction report — trajectory, turning points, risks, confidence — written into my repository. `report_plan` returns a drafting task that includes aggregated statistics the engine computed deterministically (sentiment curves per entity, engagement peaks, stance migrations, thread-level escalation chains) plus the strongest representative quotes;

the host LLM drafts the narrative; report_submit validates and versions it; report_export writes **.murmur/reports/{world}/report-N.md** with embedded Mermaid timeline and stance charts. Reports are regenerated — not mutated — so two scenarios can be diffed line by line. **Acceptance:** the report contains at minimum: most-likely trajectory, three named risks with supporting posts, a confidence section distinguishing strong-signal from contested findings, and the deterministic statistics appendix. **Parity note:** matches MiroFish's ReportAgent output, delivered as a versioned file instead of a console view.

F7 — Deep Interaction

User story. As a user, after the run I want to interrogate the world — ask any individual agent why it wrote what it wrote, and ask a ReportAgent follow-up questions over the whole simulation. interview_agent opens a focused dialogue task for one persona: the engine assembles that persona's memory, posts and trait vector as grounding, the host LLM answers in character, and the exchange is logged to the world directory. report_agent_ask runs the same pattern over the full post corpus and statistics, letting the user drill into any claim in the report (show me the posts behind this risk). **Acceptance:** an interview with any persona stays grounded — it cites its own prior posts when explaining its stance, and declines knowledge it never ingested. **Parity note:** matches MiroFish's deep-interaction step, chat-with-any-agent included.

5.8 Non-Functional Requirements

The constraints below are contract-level: violating any of them violates the product promise, so they are acceptance criteria for v1.0 as a whole, tested per client in Milestone 1.

Requirement	Target
No API key, ever	The server makes zero outbound model calls; no key field exists in any config surface; verified by network-audit test
Tool latency	P95 under 300 ms for stateless tools; under 2 s for batch composition on a 128-persona world
Fully local state	All data under .murmur/ in the workspace; no telemetry except explicit opt-in; worlds are portable by copying the directory
In-repo artifacts	Reports, graphs and interview logs written as Markdown + Mermaid; renderable on any git forge
Resumability	A crashed or closed session resumes to the exact stage boundary; no partial state
Client neutrality	Identical behavior on every MCP client in the Chapter 9 matrix; no client-specific server branches
Resource discipline	No tool response exceeds 12,000 tokens; batch sizes adapt to client-declared context limits

Table 5.1 — v1.0 non-functional requirements

6. Host-Powered Inference: The Zero-Key Architecture

6.1 The Constraint

It must work as a normal plugin or MCP server and must not require an extra LLM API key to run alongside coding agents. People should use it with the coding subscription they already pay for — Codex, Claude, Antigravity, OpenCode, Hermes — and nothing else. — founding requirement, this project

This single constraint drives every consequential design decision. MiroFish's engine is architected around server-side model calls: its process holds an `LLM_API_KEY`, spends it per token, and reaches out to Zep Cloud for memory. Wrapping that engine in an MCP skin would inherit the key requirement and the bill — so Murmur's engine is designed from zero around the opposite principle: the server never calls a model. All intelligence is borrowed from the host, through the MCP connection that already exists.

6.2 Engine and Brain

The architecture is a strict separation of engine and brain. The **Murmur engine** is a deterministic, LLM-free state machine: worlds, ontology, graph, personas, round scheduling, platform mechanics (feed ranking, threading, virality scoring), memory tiers, statistics and report templating. Given the same seeds, parameters and submitted generations, it produces byte-identical state transitions — which is what makes golden-run testing and reproducibility possible. The **brain** is the host coding agent's LLM: the same model, on the same subscription, that writes the user's code. Murmur's tools are arranged so that every task requiring intelligence is expressed as a structured prompt the host completes, and every task requiring state or arithmetic is a deterministic tool call the server performs.

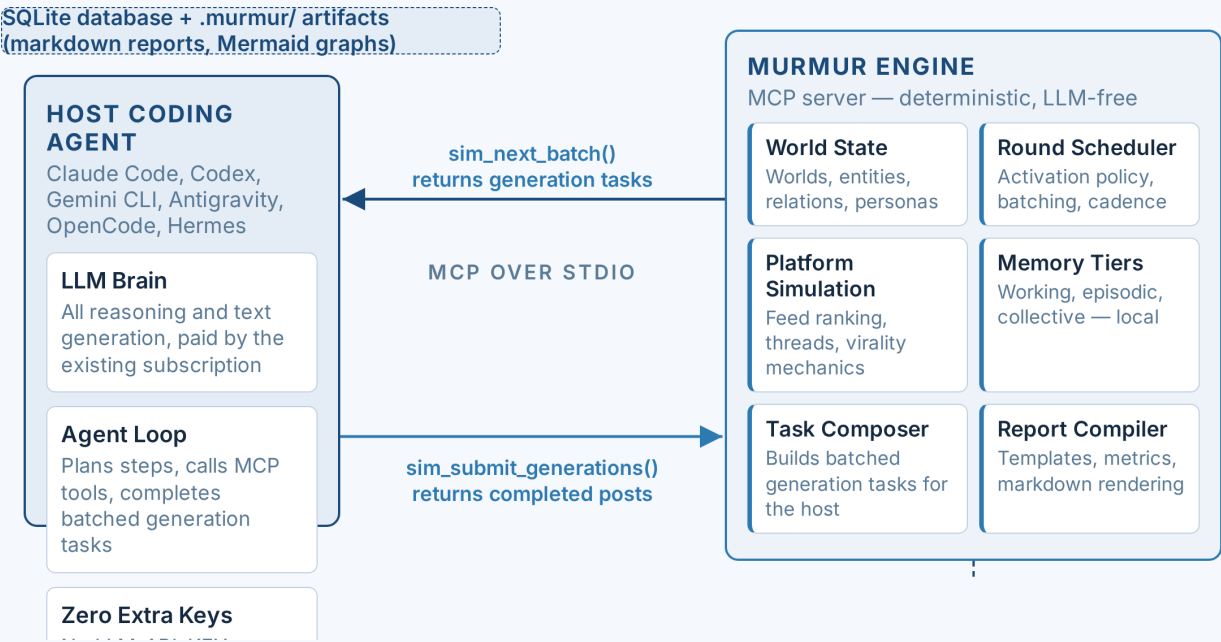


Figure 6.1 — Host-Powered Inference. The engine (right) is deterministic and LLM-free; the host agent (left) supplies all generation through batched task exchanges over MCP stdio.

6.3 The Batched Generation Protocol

The heart of the pattern is the plan / submit tool pairing. A stage begins with a plan tool — `ontology_plan`, `personas_plan`, `sim_next_batch`, `report_plan` — which returns a **generation task**: a JSON bundle containing a task id, a strict output schema, and a batch of prompt payloads composed from engine state. The host LLM completes the batch in a single response (or a few, for large batches); the matching submit tool — `ontology_submit`, `personas_submit`, `sim_submit_generations`, `report_submit` — ingests, validates against the schema, applies the results to world state, and returns a compact receipt plus a pointer to the next step. Invalid items are itemized for surgical retry, so one malformed post never re-costs an entire batch. The protocol has three properties worth naming: it is **batched** (one tool call amortizes many generations), **stateless across calls** (the engine, not the conversation, holds the world), and **auditable** (every task and submission is logged to the world directory as JSON).

8-20 posts composed per <code>sim_next_batch</code> call in the default configuration	<12k tokens in the largest tool response, enforced by the resource-discipline NFR	100% of generations performed by the host LLM on the user's existing subscription
---	--	---

6.4 Context-Budget Engineering

Borrowing the host's context window makes budget discipline a first-class engine concern, solved deterministically rather than by prompt hope. Persona digests are capped (120 tokens each) and only activated agents are included per round. Feeds are ranked and truncated per persona rather than globally, so

an agent sees the posts it would plausibly see, not the whole firehose. Between rounds the engine compacts each persona's working memory into episodic summaries; the full logs stay on disk and are re-expandable on demand for interviews. Finally, every large artifact — posts, thread trees, statistics — spills to `.murmur/` immediately, and tool responses return receipts with fetch-by-id pointers instead of inline content. The practical envelope: a 40-round, 24-persona world runs inside a single Claude Code session without context exhaustion, and larger worlds run across resumable sessions.

6.5 MCP Sampling as an Optional Accelerator

The MCP specification includes a sampling primitive that lets a server request a completion from the host client — precisely the zero-key behavior Murmur needs. Where a client implements sampling with adequate rate and permission behavior, the engine may use it to run inner-loop generations (individual posts, memory compactions) without round-tripping through the agent's outer loop. Where a client does not, the plan/submit protocol is the fallback, and because both paths write identical artifacts, behavior is client-independent. v1.0 ships with the plan/submit path as the default on every client and sampling as an opt-in accelerator flag, so the product's promises never depend on uneven client support for a newer protocol feature.

6.6 Why Not Fork MiroFish

Three independent reasons converge on the same answer. Licensing: MiroFish is AGPL-3.0, so embedding its code would force Murmur into AGPL and cloud-served-source obligations at odds with a permissive, embed-anywhere plugin strategy (Chapter 11). Architecture: the engine's model calls and Zep Cloud memory are the key-requiring center of its design — exactly what must be excised — so the deepest layers are the ones that would have to be rewritten anyway. Product shape: a Node-plus-Python web stack cannot ride an `npx` install into every coding agent. A clean-room reimplementation of the workflow concepts — stages, personas, dual platforms, reports — carries none of these liabilities, and the pipeline's concepts are not copyrightable expression. The engineering cost is a from-scratch deterministic engine, which the roadmap budgets explicitly.

7. MCP Tool Catalog v1.0

7.1 Design Principles

The tool surface follows five rules. **Deterministic core:** tools mutate state and compute; they never generate prose. **Plan/submit pairing:** every generation need is a plan tool whose result the host completes and returns through a submit tool. **Bounded responses:** no tool returns more than 12,000 tokens; larger content is written to the world directory and referenced by id. **Resumable by construction:** every tool is safe to re-issue after a crash; the engine reports the current stage and next recommended tool in every receipt. **Teaching receipts:** receipts end with a short next-step hint so a host agent that has never seen Murmur can drive the pipeline from tool descriptions alone — the property that makes prompt packs (7.3) accelerators

rather than requirements.

7.2 Tools by Group

Eighteen tools ship in v1.0, grouped below. Names are stable API; inputs and outputs are summarized — full JSON Schemas live in the repository's `packages/server` package and are exported as a machine-readable manifest for client marketplaces.

Tool	Input	Output	Notes
<code>world_init</code>	name, description	world id + stage map	Creates the world directory and SQLite store
<code>world_list</code> / <code>world_open</code>	filter / world id	worlds / stage map	Enumerate saved worlds; reopen a prior one
<code>world_status</code>	world id	stage, counts, next tool	Single source of truth for session state
<code>world_config</code>	platforms, sizes, limits	updated config	Tunes batch sizes and platform parameters
<code>seed_add_files</code>	repo-relative paths	seed digests	PDF, MD, TXT; byte-hashed dedupe
<code>seed_add_url</code> / <code>seed_add_text</code>	url / text	seed digest	Host fetches URLs and passes extracted text
<code>seeds_review</code>	world id	digest + coverage notes	Pre-flight check before ontology

Table 7.1 — World and seed management

Tool	Input	Output	Notes
<code>ontology_plan</code>	world id	generation task (batch)	Schema-strict extraction prompt bundle
<code>ontology_submit</code>	task id + JSON	entities, motives, anchors	Itemized validation; surgical retry
<code>graph_build</code>	world id	relations, tensions	Typed edges + tension scores
<code>graph_query</code>	entity / relation / depth	sub-graph	Token-budgeted traversal
<code>graph_export_mermaid</code>	world id, scope	mermaid path	Written to world directory
<code>personas_plan</code>	count, distribution	generation task	Archetypes derived from ontology
<code>personas_submit</code>	task id + JSON	population summary	Trait vectors, stances, platforms
<code>persona_inspect</code> / <code>persona_edit</code>	persona id	persona / updated	Audit and fix individuals pre-run

Table 7.2 — Ontology, graph and personas

Tool	Input	Output	Notes
<code>sim_configure</code>	rounds, rates, virality	config receipt	Default 8 rounds, max 40
<code>sim_next_batch</code>	world id	generation task	Activated agents, personalized digests
<code>sim_submit_generations</code>	task id + posts	round receipt + stats	Feed, memory, engagement updates
<code>sim_inject_event</code>	round, event text, scope	event receipt	God's-eye variable injection
<code>sim_round_summary / sim_timeline</code>	round / world id	compact readout	Sentiment curve, escalations
<code>report_plan</code>	world id, focus	drafting task	Deterministic stats + evidence pack
<code>report_submit / report_export</code>	draft / format	versioned report path	Markdown + Mermaid in <code>.murmur/</code>
<code>interview_agent</code>	persona id, question	in-character answer	Grounded in that persona's memory
<code>report_agent_ask</code>	question	cited answer	Drills into posts behind any claim

Table 7.3 — Simulation, reporting and interaction

7.3 Prompts: One-Command Workflows

MCP prompts let a client expose Murmur as first-class commands, so the full pipeline is one invocation rather than a tool dance the user must know. Four ship in v1.0: **murmur:predict** (seeds plus a question, runs the whole pipeline to a report), **murmur:simulate** (stops after N rounds for mid-run inspection and injection), **murmur:interview** (opens a dialogue with a persona or the ReportAgent), and **murmur:resume** (reopens a saved world at its stage boundary). Prompts are playbooks — they tell the host which tools to call in what order — so they stay client-portable and can be extended by template packs without server changes.

7.4 Resources

Three read-only resources let clients surface Murmur state natively: **murmur://worlds** (index), **murmur://world/{id}/state** (live stage, round, sentiment snapshot), and **murmur://world/{id}/report/{rid}** (rendered report). Resources make Murmur visible in client UIs that list resources — a small fidelity touch that also gives future GUI clients a stable surface to bind to without new protocol work.

7.5 A Worked Session

The transcript below, abbreviated for print, shows the tool-call dance for a launch-reaction prediction on Claude Code. The user speaks once; the host agent, guided by tool descriptions and teaching receipts, drives everything else.

```
User: /murmur:predict How will r/programming react to the new pricing page?

world_init(name="pricing-reaction")
seed_add_files(["pricing-page.md", "FAQ.md", "CHANGELOG.md"])
seeds_review() -> 3 seeds, 41 KB, coverage OK
ontology_plan() -> task: extract entities/motives/anchors
ontology_submit(...) -> 14 entities, 9 motives, 6 anchors
graph_build() -> 41 typed relations, 7 tensions > 0.6
graph_export_mermaid() -> .murmur/pricing-reaction/graph.mmd
personas_plan(count=24) -> task: draft population
personas_submit(...) -> 24 personas (11 reddit-lean, 9 twitter-lean, 4 both)
sim_configure(rounds=8)
rounds 1-4: sim_next_batch() -> sim_submit_generations(...)
sim_inject_event(round=5, "competitor undercuts price by 30%")
rounds 5-8: sim_next_batch() -> sim_submit_generations(...)
sim_timeline() -> sentiment curve, 2 escalation chains
report_plan() -> task: draft report (stats + evidence pack)
report_submit(...) -> report v1 stored
report_export("md") -> .murmur/pricing-reaction/report-1.md

Murmur: Report ready – 3 risks flagged; top predicted objection:
"migration guide reads as a paywall" (confidence: strong signal).
```

Example: launch-reaction run on Claude Code (abbreviated)

8. System Architecture and Data Model

8.1 Component Architecture

Murmur ships as a single process per client, launched on demand over stdio — the transport every MCP client supports without configuration. The process hosts the engine core (a pure, deterministic TypeScript module with no I/O), the MCP server layer (tool, resource and prompt handlers generated from shared Zod schemas), and a storage adapter bound to the workspace. All state lives under **.murmur/** at the repository root: a SQLite database holds the world graph, personas, posts and statistics, while reports, Mermaid exports, interview logs and the JSONL audit trail of every plan/submit exchange live as plain files. A world directory is therefore self-contained — zip it, commit it, or copy it to another machine and it opens with `world_open`. There is no daemon to babysit, no port to collide, and no server-side account, which is what allows the zero-config install promised in Chapter 9.

8.2 Data Model

Ten tables carry the world. The schema is deliberately denormalized where reads dominate (feeds, summaries) and fully normalized where auditability dominates (generations, events), with SQLite WAL mode for concurrent read access during interviews.

Table	Purpose	Key fields
worlds	One row per simulated world	id, name, stage, config, timestamps
seeds	Ingested material with provenance	id, world, type, hash, path/url, digest
entities	Ontology entities	id, world, name, type, salience, anchors[]
relations	Typed graph edges with tension	src, dst, type, weight, tension
personas	Simulated population	id, world, traits, stances, platform, activity
memories	Three-tier memory records	persona, tier (working/episodic/collective), content, round
posts	Every generated artifact	id, world, round, persona, platform, thread, body, metrics
events	God's-eye injections and stage marks	round, type, payload, scope
reports	Versioned prediction reports	id, world, version, path, focus
generations	Audit trail of plan/submit exchanges	task id, schema, submitted JSON, verdicts

Table 8.1 — Core schema (SQLite)

8.3 Simulation Engine Design

A round proceeds in five deterministic phases. **Activation:** the scheduler selects which personas act this round, weighted by activity level, current engagement heat on entities they care about, and a recency backoff that prevents one loud persona from dominating. **Digest composition:** for each activated persona the engine builds a personalized view — its stance-relevant feed slice (ranked by an engagement score combining recency, virality and stance salience), its working memory, and the current collective facts — capped at the persona token budget. **Task assembly:** digests are packed into a single batched generation task for the host. **Ingestion:** submitted posts are validated, stored, and fed through the platform mechanics — a Twitter-style ranking with retweet amplification and a Reddit-style thread tree with vote gravity, both parameterized in `world_config`. **Aggregation:** sentiment per entity, stance migration, escalation chains and engagement peaks are computed and stored as round statistics. Because phases two through five are pure functions of stored state, a golden-run test replays any recorded session and asserts byte-identical outcomes — the property that makes Marcus-the-analyst's reproducibility story true rather than aspirational.

8.4 The Local Memory Subsystem

MiroFish's use of Zep Cloud for agent memory is replaced by a three-tier local design that keeps every byte inside the repository. **Working memory** is the engine-composed digest each persona sees each round — transient, bounded, rebuilt on demand. **Episodic memory** is the per-persona log of salient events (what it posted, what it replied to, what it was injected with), with engine-computed salience scores driving compaction into summaries between rounds; full logs remain on disk and are re-expandable for interviews, which is what lets an interviewed persona cite its own prior posts. **Collective memory** is the world-level layer: the ontology graph, tension scores and factual anchors every persona can reference, updated as the simulation surfaces new claims. Nothing leaves the machine, there is no third-party quota to exhaust, and a world's memory — like everything else — is version-controlled with the repo.

9. Client Support and Distribution

9.1 The Support Matrix

Because Murmur is a single stdio MCP server, supporting a client is mostly an install-command exercise plus an integration test that records a full transcript. The matrix below is the v1.0 launch set — the MCP clients commanding the largest developer populations as of this plan, per the founding requirement that the product ride existing subscriptions (Codex, Claude, Antigravity, OpenCode, Hermes) and the wider ecosystem.

Client	Install	Notes
Claude Code	<code>claude mcp add murmur -- npx -y murmur-mcp</code>	Reference client; prompts surface as slash commands
Codex CLI	<code>codex mcp add murmur -- npx -y murmur-mcp</code>	Rides ChatGPT subscription plans
Gemini CLI	<code>gemini mcp add murmur -- npx -y murmur-mcp</code>	Uses Gemini plan quota
Antigravity	mcpServers JSON in agent config	Google agentic IDE; config template shipped
OpenCode	<code>opencode.json mcp entry (npx -y murmur-mcp)</code>	Open client; any backing subscription
Hermes	config JSON template in adapters package	Terminal-first, per founding requirement
Cursor	<code>cursor mcp add murmur -- npx -y murmur-mcp</code>	Also ships <code>.cursor/mcp.json</code> template
Windsurf	cascade MCP settings entry	Config template shipped
Cline / Continue	<code>cline_mcp_settings.json</code> / config YAML entry	VS Code family via config templates
VS Code Copilot	<code>.vscode/mcp.json</code> entry (<code>npx -y murmur-mcp</code>)	GitHub Copilot agent mode

Table 9.1 — Coding-agent client support at v1.0 launch

The engineering promise hiding in this table is **client neutrality**: the server contains zero client-specific branches. Per-client differences — how prompts are surfaced, how config is written, how sampling permissions behave — live entirely in the adapters package as install snippets and integration fixtures. A new client that speaks MCP is supported the day someone writes a 20-line adapter, and community adapters are a first-class contribution path.

9.2 Distribution Channels

Primary distribution is npm: **`npx -y murmur-mcp`** runs the latest release with no global install, which is what every install line in Table 9.1 relies on. Secondary channels follow where MCP users actually discover servers: the GitHub MCP Registry (a one-click add surface), Smithery and PulseMCP directories, and a README badge matrix showing the tested-client transcript status per release. A Homebrew tap is a post-1.0 nicety, not a launch dependency. Versioning follows semver with the tool surface as the contract: tool renames or schema breaks are major versions, and the machine-readable manifest from Chapter 7 lets marketplaces pin and diff capabilities automatically.

9.3 The Normal-Plugin Experience

The founding requirement — works like a normal plugin, no extra account — is enforced as a launch-blocking acceptance test rather than a marketing claim. A clean-machine script provisions a fresh container per client from Table 9.1, installs Murmur with the documented one-liner, and asserts all of the following: no network egress except the client's own model traffic, no key or token field anywhere in the config surface, first world initialized within two minutes of install, and a report file present in the workspace after the reference run. If any assertion fails on any launch client, the release blocks. The same script runs in CI on every pull request touching the server or adapters packages.

10. Technology Stack Decision

10.1 Options and Criteria

Three credible stacks were evaluated against the product's constraints. **Option A** is a clean-room TypeScript monorepo: the official MCP TypeScript SDK, a pure deterministic engine package, better-sqlite3 for storage, Zod for the shared schemas, published as one npx-runnable package. **Option B** is Python with FastMCP and a custom simulation core on PyPI — the ecosystem MiroFish itself inhabits, and the natural home for any future statistical modeling. **Option C** wraps or forks MiroFish's AGPL engine behind an MCP layer, inheriting its GraphRAG and simulation code. Criteria weights reflect the founding requirements: install friction and zero-key architecture fit dominate; AGPL safety matters because of Option C's existence; client compatibility, performance and maintainability complete the matrix.

Criterion (weight)	A: TypeScript	B: Python / FastMCP	C: Fork MiroFish
Install friction, npx zero-dep (25%)	5	2	1
Zero-key HPI architecture fit (20%)	5	5	1
AGPL / licensing safety (15%)	5	5	1
Client compatibility (15%)	5	4	3
Simulation performance (10%)	4	4	4
Maintainability and hiring (15%)	4	4	2
Weighted total	4.70	3.90	1.90

Table 10.1 — Scored decision matrix (1-5 per criterion; weighted total)

10.2 Recommendation

Option A — the lean TypeScript rebuild — is the recommendation, and the margin is not close. The decisive criterion is the distribution surface: the one-line npx install that every MCP client on the launch matrix already uses assumes a Node runtime, and a single-package npm distribution gives Murmur the same install friction as any shell command. Python would demand interpreter and environment management (uv, venv, or PyPI packaging for CLI entry points) on machines whose owners chose coding agents precisely to avoid that class of friction. Option C fails twice over: it drags AGPL obligations onto every distributor, and the key-holding, Zep-dependent architecture at its core is precisely the design the founding requirement forbids. TypeScript also puts the implementation language next to the reference MCP SDK, where protocol updates land first, and the deterministic engine benefits from a single-language, single-package test story. Python remains a friendly neighbor, not a foundation: if heavy statistical modeling is added post-1.0, it enters as an optional sidecar command the engine shells out to, not as the host process.

10.3 Repository Layout

```

murmur/
packages/
engine/ # deterministic simulation core (pure TS, zero I/O)
server/ # MCP server: tools, resources, prompts, stdio transport
prompts/ # playbook prompt packs exposed to host agents
adapters/ # per-client install snippets + config templates + fixtures
templates/ # scenario packs: launch, policy, crisis, finance, fiction
apps/
cli/ # optional thin CLI for humans and CI pipelines (post-1.0)
tests/
engine/ # golden-run replay tests (byte-identical assertions)
integration/ # recorded transcripts per launch client
cleanroom/ # no-egress, no-key, time-to-first-world release gate

```

Planned monorepo layout (pnpm workspaces)

11. Legal, Licensing and Monetization

11.1 Clean-Room Policy

MiroFish is AGPL-3.0, and Murmur's strategy depends on being embeddable anywhere. The policy is therefore strict and written down: no contributor copies MiroFish code, prompts, configuration or documentation into the repository; contributors who have worked with the MiroFish codebase document that familiarity, and substantive design decisions cite the public workflow description rather than implementation internals. What Murmur reuses — the five-stage pipeline pattern, the notion of persona-driven dual-platform simulation, the report-plus-interaction shape — is unpatented conceptual workflow, not copyrightable expression. The project's name, README and marketing never use the MiroFish trademark except in factual comparisons. This costs discipline but buys the permissive licensing that makes the distribution strategy legal by construction.

11.2 License

License	Adoption signal	Commercial flexibility	Defensive for core
Apache-2.0 (recommended)	Excellent — the default of infra tooling	Explicit patent grant; calm for enterprise adopters	Moderate
MIT	Excellent	Maximal simplicity	Low
AGPL-3.0	Deters corporate forks of hosted clones	High friction for embedded use	High
BSL / Elastic-style	Signals commercial intent	Converts to permissive over time	High

Table 11.1 — License options for the Murmur codebase

Apache-2.0 is the recommendation: it maximizes the adoption the strategy depends on, its patent grant removes the enterprise legal review that MIT's silence sometimes triggers, and it keeps every distribution channel — npm, registries, vendored copies inside company dotfiles — frictionless. The defensive scenarios AGPL guards against (a hosted rival cloning the repo) are better answered by velocity and community than by license fear at this stage.

11.3 Monetization: Paths and Decision Triggers

The founding choice is deliberately deferred — v1.0 ships free and open with no revenue mechanism — but the option space is mapped now so the decision is a switch, not a rebuild. Four paths are credible. **Pure OSS with sponsorship:** GitHub Sponsors and Open Collective; fits the zero-cost brand, caps upside. **Hosted tier**

for non-developers: the same engine behind a web console for users without a coding agent — notably the Priya persona's non-technical colleagues — sold per seat; the plugin stays free forever, which keeps the core promise intact. **Pro extensions:** visualization workbenches, CI integrations that gate releases on simulation reports, and long-run distributed simulations as paid add-ons. **Enterprise support:** support contracts and indemnification for companies embedding Murmur in internal tooling. Decision triggers are adoption-keyed rather than calendar-keyed: if weekly active worlds sustain four-digit numbers, the hosted tier earns a design sprint; if enterprise inbound concentrates in one vertical (policy analysis is the likeliest), the support path leads. No trigger fires before v1.0 ships.

12. Roadmap, Risks and Success Metrics

12.1 Roadmap

Milestone	Weeks	Scope and exit criteria
M0 — Walking Skeleton	1-4	Engine core (state, round loop, platform mechanics) behind world_init/seed/ontology/graph tools; happy path end-to-end on Claude Code; golden-run harness live. Exit: /murmur:predict produces a report from a PDF seed.
M1 — Multi-Client Beta	5-8	Personas, simulation and injection tools complete; five launch clients integrated with recorded transcripts; npm 0.x published; cleanroom gate script green. Exit: identical run on Claude Code, Codex and Gemini CLI.
M2 — Parity and Quality	9-14	Report v2 with deterministic statistics appendix; interviews and report_agent_ask; Mermaid exports; world diff between runs; template packs (launch, policy, crisis, fiction). Exit: full F1-F7 acceptance criteria green.
M3 — v1.0 and Ecosystem	15-24	Registry listings and marketplace manifests; documentation site; opt-in telemetry (run counts only); remaining launch clients; 1.0 semver contract. Exit: clean-machine install test passes on all Table 9.1 clients.

Table 12.1 — Six-month roadmap to v1.0

12.2 Risk Register

Risk	L	I	Mitigation
Subscription rate limits throttle long simulations	M	H	Batching amortizes calls (8-20 posts per call); round checkpoints make runs resumable across sessions; guidance tunes rounds to plan quotas
Host context window exhaustion on large worlds	M	H	Chapter 6.4 budget engineering is an engine contract, tested by golden runs up to 40 rounds x 128 personas
Uneven MCP sampling support across clients	H	M	Sampling is opt-in acceleration only; plan/submit is the universal default path with identical artifacts

Risk	L	I	Mitigation
Persona behavior drifts flat or repetitive	M	M	Stance vectors + tension scores drive activation; prompt packs A/B-tested in M2; interviews expose low-fidelity personas pre-run
AGPL proximity claim from code similarity	L	H	Clean-room policy with documented provenance (11.1); CI grep against MiroFish source strings as a tripwire
MiroFish ships its own MCP server	M	M	It would still carry its key/bill architecture; Murmur's differentiation is economic and workflow-native, not feature parity alone
Prompt injection via malicious seed material	M	M	Seeds enter engine state as data, never as instructions; generation tasks quote seeds inside delimiters; submission schemas reject instruction-like output fields

Table 12.2 — Risk register (L = likelihood, I = impact, graded High / Medium / Low)

12.3 Success Metrics

Metrics follow the adoption funnel the strategy depends on, instrumented only through opt-in run-count telemetry and public signals — no content ever leaves the machine. **Activation:** time from install to first initialized world under ten minutes for the median clean-machine run (the cleanroom gate measures this continuously). **Engagement:** worlds created per week, share of worlds that reach a report, and share of reports followed by at least one interview — the last being the truest signal that deep interaction, the stage web-app competitors treat as a demo, is pulling weight as a product surface. **Distribution:** coverage of the Table 9.1 client matrix with green transcripts per release, plus npm download trend and GitHub star trajectory benchmarked honestly against MiroFish's own growth curve. **Community:** external template-pack and adapter contributions, the earliest predictor of the ecosystem moat this plan is actually building.

<10 min median install-to-first-world time, enforced by the release gate	100% of launch clients passing the clean-machine test at 1.0	6 mo from kickoff to v1.0 on the committed roadmap
--	---	---

12.4 Immediate Next Steps

- Ratify this plan: confirm the codename direction, the Apache-2.0 recommendation and the deferred-monetization triggers,
- Stand up the monorepo skeleton (packages per 10.3) with the golden-run harness in CI from commit one,
- Ship M0's walking skeleton to a private cohort of Claude Code and Codex users and let the first real report file decide whether the world feels as promised.